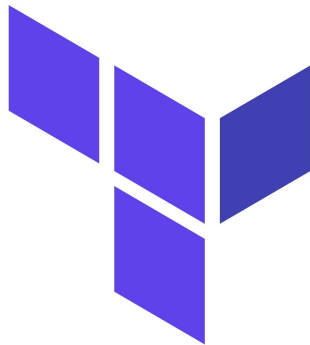




Tesina Finale di
Virtual Networks and Cloud Computing
Corso di Laurea Magistrale in Ingegneria Informatica e Robotica – A.A. 2025-2026
DIPARTIMENTO DI INGEGNERIA

Docente
Prof. Gianluca REALI



Terraform per Kubernetes

Analisi tecnica delle funzionalità di Terraform applicate alla gestione di cluster Kubernetes.

Studente

372969 **Luca Fagiolo** luca.fagiolo@studenti.unipg.it
Repository Github: https://github.com/Lukzard9/Project_VNCC

0. Indice

1	Introduzione	2
1.1	Obiettivi	2
2	Setup del cluster	4
2.1	Installazione Kubernetes, Kind e Terraform	4
3	Avvio del cluster e distribuzione della prima infrastruttura	6
4	Ambienti modulari e parametrizzazione	10
4.1	Root Module (Livello superiore)	11
4.2	Child Module (Livello inferiore)	12
5	Gestione Dinamica di Credenziali e Certificati TLS	13
5.1	Generazione dinamica di password e gestione Secret	13
5.2	Creazione di certificati TLS Self-Signed	14
5.3	Iniezione delle configurazioni e risoluzione delle dipendenze	15
5.4	Esportazione della configurazione con il provider Local	16
6	Pipeline Completa e Controllo del Flusso	17
6.1	Creazione del cluster e Provider Chaining	17
6.2	Gestione delle Dipendenze	18
6.3	Esportazione dei Dati	19
7	Funzionalità Terraform non analizzate	20
7.1	Gestione Remota dello Stato	20
7.2	Infrastrutture Multi-Cloud	20
7.3	Regole di Sicurezza Automatiche	21
7.4	Importazione di Infrastrutture Esistenti	21
8	Bibliografia	22

1. Introduzione

Terraform è uno strumento open source sviluppato da HashiCorp [1] che rappresenta una soluzione moderna per la creazione e la gestione automatizzata delle infrastrutture informatiche. Terraform utilizza file di configurazione testuali per dichiarare lo stato finale desiderato del sistema, incaricandosi poi di comunicare istantaneamente con i servizi coinvolti per istanziare le risorse necessarie.

Questa metodologia, nota come Infrastructure as Code (IaC), nasce per superare i pesanti limiti degli approcci tradizionali di gestione infrastrutturale. Nelle metodologie classiche l'allocazione e la configurazione dei server e delle reti avviene spesso in modo manuale, interagendo direttamente con le interfacce grafiche dei provider. Queste pratiche presentano notevoli svantaggi temporali e organizzativi, infatti l'infrastruttura risulta macchinosa da replicare o scalare, le modifiche apportate faticano a essere documentate e tracciate in modo affidabile, e il rischio di incongruenze (*configuration drift*) tra i vari ambienti applicativi aumenta in maniera considerevole. Al contrario l'IaC permette di centralizzare la configurazione architetturale riconducendola interamente alla scrittura e al versionamento di codice. In questo modo si limitano i potenziali errori umani e le problematiche di scalabilità, garantendo la totale automazione ed elevata riproducibilità degli ambienti.

Per questi motivi Terraform si integra perfettamente con Kubernetes [2]. Anziché dover compilare e mantenere molteplici file YAML separati in maniera manuale, Terraform consente di gestire il cluster e le sue risorse con un approccio centralizzato calcolando automaticamente le dipendenze tra i vari componenti e offrendo l'importante vantaggio di visualizzare un'anteprima esatta e sicura delle modifiche prima che vengano confermate.

1.1 Obiettivi

L'obiettivo principale di questo progetto consiste nel dimostrare l'utilizzo pratico di Terraform per l'amministrazione efficiente di risorse all'interno di un cluster Kubernetes. Non avendo a disposizione un fornitore cloud reale, come ad esempio AWS o Google Cloud, si è optato per la simulazione dell'intero ambiente su una

macchina locale avvalendosi di Kind [3] , uno strumento che esegue i nodi di Kubernetes all'interno di normali container Docker.

Nei prossimi capitoli si mostrerà il setup iniziale dell'ambiente Kind e la creazione di una prima semplice infrastruttura di base. Questo passaggio servirà a mostrare il collegamento automatico tra le risorse e l'utilità del comando **terraform plan** per visualizzare in anticipo le modifiche.

Successivamente si passerà a scenari più avanzati. Verrà mostrato come usare i moduli e i cicli per creare configurazioni facilmente riutilizzabili per ambienti separati (come sviluppo o test). Si vedrà poi come automatizzare la sicurezza, generando password e certificati da inserire direttamente nei container senza effettuare passaggi manuali. Infine l'ultimo capitolo illustrerà come gestire l'intero ciclo di vita del progetto, dalla definizione automatica del cluster fino all'esportazione automatica di tutti quei dati essenziali per l'interazione con l'architettura finale.

2. Setup del cluster

Generalmente simulare in locale un cluster Kubernetes multi-nodo richiede macchine virtuali pesanti, come VirtualBox. Questo approccio consuma molte risorse e spesso rende macchinosa e complessa la configurazione della rete. Per questo motivo si è deciso di utilizzare Kind (Kubernetes in Docker), che è una tecnologia che semplifica il tutto eseguendo i nodi Kubernetes, sia control-plane che worker, come container Docker leggeri.

In combinazione con WSL2, Kind permette di creare un ambiente affidabile e facilmente riproducibile. Questa architettura è particolarmente utile per mostrare le funzionalità di Terraform perché replica in modo realistico i flussi di lavoro cloud-native.

Se un deployment fallisce oppure lo stato del cluster cambia in modo imprevisto l'intero ambiente può essere eliminato e ricreato in pochi secondi. In questo modo risulta più chiaro il funzionamento di Terraform, soprattutto per quanto riguarda la gestione dello stato e l'idempotenza.

2.1 Installazione Kubernetes, Kind e Terraform

Si presuppone di avere già installato e configurato l'ultima versione di WSL2 e Docker Desktop. Entrati nel terminale WSL2 basta seguire la [guida ufficiale](#) per l'installazione di Kubernetes.

```
1 sudo apt-get update
2 sudo apt-get install -y apt-transport-https ca-certificates curl
  gnupg
3
4 # If the folder '/etc/apt/keyrings' does not exist, it should be
  created before the curl command, read the note below.
5 # sudo mkdir -p -m 755 /etc/apt/keyrings
6 curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.36/deb/Release.key
  | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-
  keyring.gpg
7 sudo chmod 644 /etc/apt/keyrings/kubernetes-apt-keyring.gpg #
  allow unprivileged APT programs to read this keyring
8
```

```

9 # This overwrites any existing configuration in /etc/apt/sources.
  list.d/kubernetes.list
10 echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
     https://pkgs.k8s.io/core:/stable:/v1.36/deb/ /' | sudo tee /
     etc/apt/sources.list.d/kubernetes.list
11 sudo chmod 644 /etc/apt/sources.list.d/kubernetes.list # helps
     tools such as command-not-found to work correctly
12
13 sudo apt-get update
14 sudo apt-get install -y kubectl

```

Per l'installazione di Kind sono necessari i seguenti comandi.

```

1 # For AMD64 / x86_64
2 [ $(uname -m) = x86_64 ] && curl -Lo ./kind https://kind.sigs.k8s.
   io/dl/v0.31.0/kind-linux-amd64
3 # For ARM64
4 [ $(uname -m) = aarch64 ] && curl -Lo ./kind https://kind.sigs.k8s.
   io/dl/v0.31.0/kind-linux-arm64
5 chmod +x ./kind
6 sudo mv ./kind /usr/local/bin/kind

```

Infine per l'installazione di Terraform sono stati eseguiti i seguenti comandi.

```

1 sudo apt-get update && sudo apt-get install -y gnupg software-
   properties-common
2
3 wget -O- https://apt.releases.hashicorp.com/gpg | \
4 gpg --dearmor | \
5 sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg > /dev/
   null
6
7 echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/
   keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.
   hashicorp.com $(grep -oP '(?<=UBUNTU_CODENAME=).*' /etc/os-
   release || lsb_release -cs) main" | sudo tee /etc/apt/sources.
   list.d/hashicorp.list
8
9 sudo apt update
10 sudo apt-get install terraform

```

3. Avvio del cluster e distribuzione della prima infrastruttura

Per verificare il corretto funzionamento dell'ambiente è stata distribuita una prima infrastruttura Kubernetes su un cluster Kind. Anche se si tratta di un'architettura molto semplice, l'utilizzo di Terraform al posto dei tradizionali manifest YAML introduce alcuni vantaggi ottimizzando l'intero flusso di lavoro.

Ad esempio l'utilizzo di riferimenti dinamici previene la duplicazione del codice. Mentre nei file YAML è spesso necessario ripetere le stesse informazioni in svariate configurazioni (come etichette o nomi di risorse), Terraform riutilizza questi dati dinamicamente. Quindi il Service all'interno del codice non richiede l'inserimento manuale del selettore, ma utilizza in automatico le etichette generate dal Deployment.

```
1 resource "kubernetes_service" "nginx_svc" {
2   # ...
3   spec {
4     selector = kubernetes_deployment.nginx.metadata.0.labels
5     # ...
6   }
7 }
```

Questo principio risulta fondamentale anche per l'assegnazione di parametri e volumi. Nel Deployment, i riferimenti a Secret e ConfigMap puntano direttamente alle risorse Terraform.

```
1     env {
2       name = "DB_PASSWORD"
3       value_from {
4         secret_key_ref {
5           name = kubernetes_secret.app_secret.metadata.0.
name
6           key = "password"
7         }
8       }
9     }
```

In caso di variazione nel nome del componente originario Terraform propaga la modifica su tutte le risorse dipendenti. Questo meccanismo gestisce implicitamente anche l'ordine di creazione, infatti Terraform deduce dal grafo delle dipendenze che il Deployment necessita della ConfigMap e del Secret e provvede a crearli per primi.

Un'ulteriore funzionalità di controllo è il blocco `lifecycle` [4] all'interno della definizione del `PersistentVolumeClaim` (PVC). Il costrutto `lifecycle` permette di alterare il comportamento standard di creazione, aggiornamento e distruzione delle risorse gestito da Terraform. Ad esempio, l'attributo `prevent_destroy` viene usato per impedire l'eliminazione accidentale di risorse critiche, come i volumi contenenti dati persistenti.

```
1 resource "kubernetes_persistent_volume_claim" "data_pvc" {
2   wait_until_bound = false
3   metadata {
4     name = "data-pvc"
5   }
6   # ...
7   lifecycle {
8     prevent_destroy = false
9   }
10 }
```

Infine, il comando `terraform plan` offre in anticipo un resoconto esatto di ciò che verrà creato, modificato o distrutto, agendo come rete di sicurezza prima di confermare l'esecuzione.

Il progetto Terraform si divide in quattro file principali:

1. **providers.tf**: Definisce il provider Kubernetes [5] e lo autentica leggendo il `kubeconfig` locale.

```
1 provider "kubernetes" {
2   config_path    = "~/.kube/config"
3   config_context = "kind-kind"
4 }
```

2. **config.tf**: Ospita le istanze passive usate dai Pod, come Secret, ConfigMap e PersistentVolumeClaim.
3. **main.tf**: Definisce l'architettura principale (in questo caso il Deployment per le repliche di Nginx), allocando dinamicamente volumi e variabili d'ambiente.
4. **network.tf**: Contiene il NodePort Service connesso al Deployment.

Per inquadrare l'architettura di questo primo esempio si riporta lo schema logico dell'ambiente.

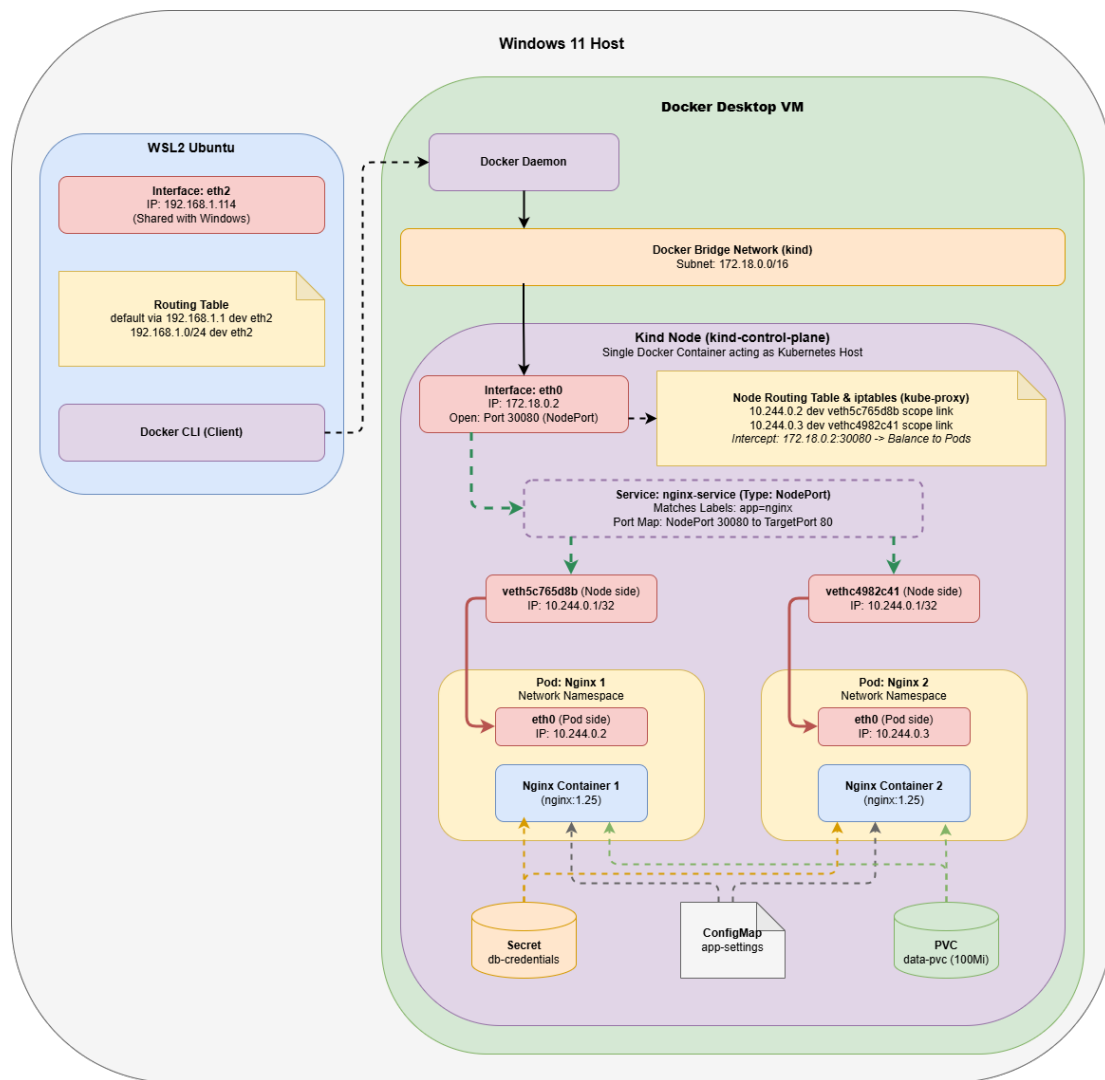


Figura 3.1: Grafico della prima versione del cluster

Per avviare l'infrastruttura si inizia lanciando il container "kind-control-plane" dal terminale WSL2 tramite il comando `kind create cluster`. Successivamente è possibile lanciare i seguenti comandi per inizializzare e applicare l'architettura:

```
1 terraform init
2 terraform plan
3 terraform apply
```

Al termine di `terraform apply`, Terraform genera i file di stato `terraform.tfstate` e `terraform.tfstate.backup`. Il primo rappresenta uno snapshot dell'infrastruttura attuale e viene usato da Terraform per rilevare varia-

zioni nello stato reale; il secondo conserva un backup per l'eventuale ripristino in caso di malfunzionamenti. Il comando `terraform plan` è in grado di sfruttare questo stato per rilevare in autonomia qualsiasi drift architetturale rispetto all'ultima esecuzione, segnalando la modifica da riconciliare. Di seguito è riportato un esempio di utilizzo di `terraform plan` tramite l'output del terminale.

```
(nl_env) user@WINDELL-HRIRMOB: /vacc/first_cluster$ kubectl set image deployment/nginx-deployment nginx=nginx:1.24
deployment.apps/nginx-deployment image updated
(nl_env) user@WINDELL-HRIRMOB: /vacc/first_cluster$ terraform plan
kubernetes_secret.app_secret: Refreshing state... [id=default/db-credentials]
kubernetes_config_map.app_config: Refreshing state... [id=default/app-settings]
kubernetes_persistent_volume_claim.data_pvc: Refreshing state... [id=default/data-pvc]
kubernetes_deployment.nginx: Refreshing state... [id=default/nginx-deployment]
kubernetes_service.nginx_svc: Refreshing state... [id=default/nginx-service]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  ~ update in-place

Terraform will perform the following actions:

# kubernetes_deployment.nginx will be updated in-place
~ resource "kubernetes_deployment" "nginx" {
  id      = "default/nginx-deployment"
  # (1 unchanged attribute hidden)

  ~ spec {
    # (5 unchanged attributes hidden)

    ~ template {
      ~ spec {
        # (18 unchanged attributes hidden)

        ~ container {
          ~ image = "nginx:1.24" -> "nginx:1.25"
            name   = "nginx"
            # (9 unchanged attributes hidden)

            # (4 unchanged blocks hidden)
        }
      }
    }
  }
  # (2 unchanged blocks hidden)
```

Figura 3.2: Output di `terraform plan` dopo una modifica manuale al cluster

4. Ambienti modulari e parametrizzazione

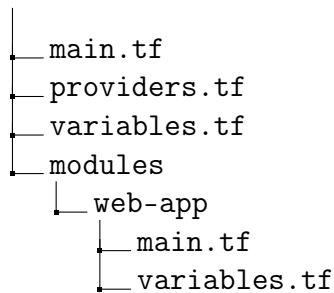
In questo capitolo si mostrano funzionalità di Terraform più avanzate come l'isolamento degli ambienti tramite cicli iterativi, la scomposizione del codice in moduli riutilizzabili [6] e la parametrizzazione avanzata.

In particolare, il ciclo `for_each` [7] di Terraform scorre una lista di stringhe contenente i nomi degli ambienti desiderati e per ogni elemento crea in modo automatico un Namespace all'interno del cluster. Successivamente lo stesso ciclo viene impiegato per invocare il modulo personalizzato passando come argomenti le variabili specifiche per ciascun contesto. All'interno del modulo la logica condizionale esamina il nome dell'ambiente attivo e modifica dinamicamente le proprietà delle risorse, riducendo ad esempio il numero di repliche dei pod nel Namespace "Dev" per ottimizzare il consumo di memoria e risorse di calcolo.

L'implementazione di una simile strategia di isolamento tramite gli strumenti nativi di Kubernetes risulterebbe notevolmente più complessa. Infatti gli amministratori di sistema sarebbero costretti a duplicare manualmente i file di configurazione YAML per ogni singolo ambiente, aumentando sensibilmente il rischio di errori di battitura e disallineamenti. Inoltre come già detto precedentemente, Kubernetes non possiede un meccanismo nativo per tracciare lo stato delle risorse create, rendendo difficoltosa la rilevazione delle modifiche non autorizzate e la successiva riconciliazione automatica.

La directory principale del progetto ospita i file di configurazione generali come la definizione dei provider e il *root module*. Questo livello superiore coordina l'infrastruttura globale e richiama i *child module* personalizzati che raggruppano logicamente le risorse Kubernetes correlate come deployment, servizi e volumi. Parallelamente i file delle variabili contengono i parametri dinamici che modificano l'allocazione delle risorse in base all'ambiente di destinazione.

Di seguito si analizzano nel dettaglio le componenti principali di questo cluster, esaminando in modo più mirato i frammenti di codice più significativi.



4.1 Root Module (Livello superiore)

Nel file delle variabili di livello superiore (`variables.tf`), viene dichiarata la lista degli ambienti per i quali si vuole replicare l'infrastruttura.

```
1 variable "environments" {
2     type      = set(string)
3     default   = ["dev", "test", "qa"]
4 }
```

Nel file principale del modulo root (`main.tf`) Terraform scorre iterativamente la variabile precedente utilizzando il loop `for_each`. La prima operazione consiste nella creazione automatica di un Namespace dedicato per ciascun ambiente nella lista.

```
1 resource "kubernetes_namespace" "envs" {
2     for_each = var.environments
3
4     metadata {
5         name = "myapp-${each.key}"
6     }
7 }
8 # ...
```

Una volta assicurato l'isolamento a livello di cluster Kubernetes, lo stesso meccanismo iterativo viene impiegato per istanziare il modulo child personalizzato (`web-app`). Qua si evidenzia l'iniezione dinamica dei parametri, infatti l'argomento `replicas` viene modulato tramite logica condizionale per limitare i pod nel caso dell'ambiente dev (1 sola replica) e garantendo 2 repliche per gli altri ambienti.

```
1 # ...
2 module "nginx_infrastructure" {
3     source      = "../modules/web-app"
4     for_each    = var.environments
5
6     namespace   = kubernetes_namespace.envs[each.key].metadata.0.
        name
```

```

7   environment = each.key
8   app_name    = "frontend"
9
10  replicas     = each.key == "dev" ? 1 : 2
11 }

```

4.2 Child Module (Livello inferiore)

Spostando l'attenzione all'interno della sottocartella, il file `variables.tf` espone i parametri di input necessari alla personalizzazione di una singola istanza infrastrutturale. Ad esempio la definizione del campo `replicas` è la seguente.

```

1 # ...
2 variable "replicas" {
3   description = "Numero di pod"
4   type        = number
5   default     = 2
6 }
7 # ...

```

Infine il file `main.tf` usa le variabili in ingresso per configurare dinamicamente le risorse. Di seguito questo frammento del `Deployment` mostra come i valori passati definiscano automaticamente namespace, labels e numero di replicas.

```

1 # ...
2 resource "kubernetes_deployment" "app_deploy" {
3   metadata {
4     name      = "${var.app_name}-deployment"
5     namespace = var.namespace
6     labels    = { app = var.app_name, env = var.environment }
7   }
8
9   spec {
10    replicas = var.replicas
11  }
12 # ...

```

Questo meccanismo garantisce coerenza tra componenti come ConfigMap e volumi, nominandoli a partire dalle stesse variabili. L'uso combinato dei moduli e del ciclo `for_each` rende così l'intera architettura altamente scalabile. Infatti per aggiungere un nuovo ambiente è sufficiente inserire il suo nome nella lista `environments` senza dover modificare manualmente alcun file di configurazione. Inoltre la gestione centralizzata dei parametri tramite variabili consente di adattare facilmente l'allocazione delle risorse in base alle esigenze specifiche di ciascun contesto, ottimizzando l'efficienza complessiva del cluster.

5. Gestione Dinamica di Credenziali e Certificati TLS

Una delle caratteristiche fondamentali di Terraform è la sua estensibilità tramite l'uso dei **Provider**. Un provider è essenzialmente un plugin che traduce le direttive scritte in linguaggio HCL in modo da chiamare le API corrette del sistema di destinazione. Terraform utilizza questi plugin non solo per comprendere le risorse disponibili e gestirne l'intero ciclo di vita, ma anche per interfacciarsi con un ecosistema tecnologico vasto e diversificato. Questa capacità permette a Terraform di gestire simultaneamente piattaforme molto diverse tra loro, come ad esempio: provider cloud (AWS, Google Cloud o Azure), sistemi di orchestrazione (Kubernetes), servizi SaaS (GitHub o Cloudflare), sistemi di monitoraggio (Datadog) o persino database e costrutti generici locali.

Proprio grazie alla combinazione di molteplici provider (come `random`, `tls`, `kubernetes` e `local`), nei prossimi paragrafi si mostrerà come generare certificati, password e file di configurazione in totale autonomia.

5.1 Generazione dinamica di password e gestione Secret

In un ambiente Kubernetes la gestione di dati sensibili e certificati risulta spesso complessa. Generalmente richiede l'uso di script Bash con openssl, laboriose codifiche manuali in Base64 per i Secret, oppure l'installazione di controller aggiuntivi come `cert-manager`. Questa configurazione mostra invece come Terraform possa generare in autonomia password sicure ed inserirle direttamente in un Secret Kubernetes senza l'intervento manuale dell'utente.

```
1 resource "random_password" "db_password" {
2   length      = 16
3   special     = true
4   override_special = " !#$%&*() -_+[]{}<>:?"
5 }
6
7 resource "kubernetes_secret" "app_secret" {
```

```

8  metadata {
9    name = "db-credentials"
10 }
11 data = {
12   "password" = random_password.db_password.result
13 }
14 type = "Opaque"
15 }

```

Tramite il provider `random` [8], viene generata una stringa di 16 caratteri. Il valore risultante (`random_password.db_password.result`) viene referenziato immediatamente dal blocco `kubernetes_secret`. Kubernetes richiede normalmente che i valori all'interno dei Secret YAML siano preventivamente codificati in Base64. Invece Terraform gestisce automaticamente la codifica e garantisce un livello di astrazione maggiore, minimizzando l'esposizione di informazioni sensibili in file testuali.

5.2 Creazione di certificati TLS Self-Signed

Similmente alla generazione della password, il provider `tls` [9] permette di generare chiavi private asimmetriche (come RSA) e certificati autofirmati senza dipendere da binaries o tool di sistema esterni.

```

1  resource "tls_private_key" "nginx_key" {
2    algorithm = "RSA"
3    rsa_bits  = 2048
4  }
5
6  resource "tls_self_signed_cert" "nginx_cert" {
7    private_key_pem = tls_private_key.nginx_key.private_key_pem
8
9    subject {
10     common_name = "localhost"
11     organization = "Demo Terraform IaC"
12   }
13   validity_period_hours = 8760 # 1 anno
14   allowed_uses = [
15     "key_encipherment",
16     "digital_signature",
17     "server_auth",
18   ]
19 }
20
21 resource "kubernetes_secret" "tls_secret" {
22   metadata {
23     name = "nginx-tls"
24   }

```

```

25 data = {
26     "tls.crt" = tls_self_signed_cert.nginx_cert.cert_pem
27     "tls.key" = tls_private_key.nginx_key.private_key_pem
28 }
29 type = "kubernetes.io/tls"
30 }

```

Il certificato e la chiave generati vengono inseriti dinamicamente in un secret di tipo `kubernetes.io/tls`, pronti per essere montati ed utilizzati dal proxy Nginx. Questo approccio elimina i passaggi manuali che risultano particolarmente rischiosi e tediosi, evitando anche la necessità di configurare moduli Kubernetes aggiuntivi in un cluster locale o di test.



```

(ml env) user@WINDELL-HR1RMUB: /vncc/randomness_certificate$ curl https://localhost:30443
curl: (60) SSL certificate problem: self-signed certificate
More details here: https://curl.se/docs/sslcerts.html

curl failed to verify the legitimacy of the server and therefore could not
establish a secure connection to it. To learn more about this situation and
how to fix it, please visit the web page mentioned above.
(ml env) user@WINDELL-HR1RMUB: /vncc/randomness_certificate$ curl -k https://localhost:30443
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>

```

Figura 5.1: Verifica del certificato TLS generato da Terraform

5.3 Iniezione delle configurazioni e risoluzione delle dipendenze

L'integrazione di queste risorse all'interno di un Deployment evidenzia l'utilità del Dependency Graph di Terraform. Il `kubernetes_deployment` che istanzia Nginx può referenziare dinamicamente i nomi e le proprietà sia del Secret contenente la password, sia dei volumi con i certificati in essi contenuti.

```

1 # ...
2 env {
3     name = "DB_PASSWORD"
4     value_from {
5         secret_key_ref {
6             name = kubernetes_secret.app_secret.metadata.0.name

```



```

7     key   = "password"
8   }
9 }
10 }
11 # ...
12 volume {
13   name = "tls-certs"
14   secret {
15     secret_name = kubernetes_secret.tls_secret.metadata.0.name
16   }
17 }
18 # ...

```

Se il nome o altre proprietà del Secret dovessero mutare, ad esempio per l'aggiunta di un prefisso o suffisso dovuto ad un nuovo ambiente, Terraform propagerà automaticamente la modifica aggiornando la definizione nel Deployment e all'occorrenza forzandone il riavvio per caricare il nuovo valore. Con Kubernetes standalone eventuali disallineamenti tra i vari file YAML verrebbero riscontrati solamente a runtime con pod bloccati in stato di errore. Terraform scongiura questi scenari procedendo a un controllo di validazione a priori e stabilendo correttamente l'ordine di rilascio tramite le dipendenze dichiarate.

5.4 Esportazione della configurazione con il provider Local

Oltre alla gestione di risorse remote, Terraform è in grado di interagire con il file system della macchina utente attraverso il provider `local` [10]. Questo risulta particolarmente utile per automatizzare la creazione di file di configurazione, eliminando il copia-incolla manuale o passaggi ausiliari per il completamento del setup.

Ad esempio, Terraform può utilizzare i dati appena generati e le info del cluster per orchestrare un file `.env` tramite la risorsa `local_file`:

```

1 resource "local_file" "app_env_file" {
2   filename = "${path.module}/.env"
3
4   content = <<-EOT
5     APP_ENV=development
6     DB_PASSWORD=${random_password.db_password.result}
7     KIND_API_ENDPOINT=${kind_cluster.local_cluster.endpoint}
8     APP_HTTP_URL=http://localhost:30080
9     APP_HTTPS_URL=https://localhost:30443
10  EOT
11 }

```

6. Pipeline Completa e Controllo del Flusso

In questo capitolo vengono illustrati i passaggi che permettono a Terraform di automatizzare la gestione dell'intera infrastruttura. Rispetto all'uso di script separati e comandi manuali, Terraform offre un controllo diretto su tutte le fasi del ciclo di vita di un'applicazione.

6.1 Creazione del cluster e Provider Chaining

Finora il cluster Kind è stato creato a mano dal terminale tramite un comando CLI, ma questo approccio va in contrasto con le vere potenzialità dell'Infrastructure as Code. Perciò in questo nuovo esempio viene usato il provider `kind` [11] per far generare il cluster direttamente a Terraform.

```
1 resource "kind_cluster" "local_cluster" {
2   name          = "tf-kind-cluster4"
3   wait_for_ready = true
4
5   kind_config {
6     kind      = "Cluster"
7     api_version = "kind.x-k8s.io/v1alpha4"
8
9     node {
10      role = "control-plane"
11      extra_port_mappings {
12        container_port = 30080
13        host_port      = 30080
14        protocol       = "TCP"
15      }
16    }
17  }
18 }
```

Il beneficio principale di questo approccio prende il nome di *Provider Chaining*. Come si può vedere nel file `providers.tf`, il provider di Kubernetes non

utilizza più un file di configurazione locale ma recupera le credenziali in memoria direttamente dal cluster appena creato.

```
1 provider "kubernetes" {
2   host          = kind_cluster.local_cluster.endpoint
3   client_certificate = kind_cluster.local_cluster.
  client_certificate
4   client_key      = kind_cluster.local_cluster.client_key
5   cluster_ca_certificate = kind_cluster.local_cluster.
  cluster_ca_certificate
6 }
```

Grazie a questa catena logica il solo comando `terraform apply` è in grado di avviare il cluster, ottenere i certificati ed eseguire i workload in modo completamente automatico e sequenziale.

6.2 Gestione delle Dipendenze

Su Kubernetes è spesso difficile imporre l'ordine esatto di avvio dei Pod. Per esempio può capitare di dover effettuare una migrazione del database prima di far avviare il server web. Terraform risolve facilmente il problema usando dipendenze esplicite.

Prendiamo in esame il Job di migrazione definito nel file `job.tf`:

```
1 resource "kubernetes_job" "db_migration" {
2   metadata {
3     name = "db-migration-job"
4   }
5   spec {
6     template {
7       metadata {}
8       spec {
9         container {
10          name      = "migrator"
11          image     = "busybox:latest"
12          command = ["sh", "-c", "echo 'Esecuzione migrazione DB
in corso...'; sleep 15; echo 'Migrazione completata'"]
13        }
14        restart_policy = "Never"
15      }
16    }
17  }
18  wait_for_completion = true
19 }
```

La regola `wait_for_completion = true` comunica a Terraform di bloccare la pipeline finché il Job non è andato a buon fine. Nel file `main.tf`, il Deployment

relativo a Nginx aspetterà pazientemente l'esito della task grazie all'argomento `depends_on`:

```
1 resource "kubernetes_deployment" "nginx" {
2   depends_on = [kubernetes_job.db_migration]
3   # ...
4 }
```

6.3 Esportazione dei Dati

L'ultimo aspetto analizzato riguarda l'esportazione dei dati dell'infrastruttura, come porte e indirizzi IP. Nel file `outputs.tf` vengono mostrati i due strumenti principali: i Data Source [12] e gli Output [13].

```
1 data "kubernetes_namespace" "default" {
2   metadata {
3     name = "default"
4   }
5   depends_on = [kind_cluster.local_cluster]
6 }
7
8 output "cluster_uid" {
9   description = "L'UID univoco del cluster Kind"
10  value       = data.kubernetes_namespace.default.metadata[0].uid
11 }
12
13 output "nginx_access_port" {
14   description = "La porta NodePort per accedere a Nginx dal
15   browser"
16   value       = kubernetes_service.nginx_svc.spec[0].port[0].
17   node_port
18 }
```

I Data Source (in questo caso `kubernetes_namespace`) si usano per leggere e caricare informazioni di risorse già esistenti sul cluster. Gli Output, invece, stampano un valore utile a schermo a fine esecuzione. In questo frammento di codice si recuperano e stampano l'Identificatore Univoco del nodo (UID) e la NodePort usata da Nginx. L'esportazione in output è un passaggio chiave poiché permette ai sistemi automatizzati (ad esempio le pipeline CI/CD) di ottenere dinamicamente le variabili di cui hanno bisogno senza compiere estrazioni manuali.

7. Funzionalità Terraform non analizzate

Nei capitoli precedenti sono stati illustrati esempi pratici basati su un cluster locale simulato tramite Kind. Tuttavia l'uso di Terraform in ambienti ospitati su cloud reali offre funzionalità molto più potenti. Di seguito vengono descritte alcune di queste caratteristiche non testate in locale.

7.1 Gestione Remota dello Stato

Negli esempi mostrati Terraform ha salvato l'elenco delle risorse create all'interno di un file locale (`terraform.tfstate`). Questo approccio risulta estremamente limitato e poco sicuro quando si lavora in team. Per questo motivo Terraform permette di archiviare il file di stato su servizi cloud remoti come Amazon S3, Azure Blob Storage o HashiCorp Cloud [\[14\]](#). La gestione remota offre backup automatizzato e un meccanismo essenziale chiamato *State Locking*. Questo sistema blocca il file di stato non appena un utente avvia un'operazione di pianificazione o applicazione, impedendo agli altri membri del team di effettuare modifiche concorrenti che causerebbero la corruzione irreversibile delle dipendenze architetturali e perdite di consistenza del cluster.

7.2 Infrastrutture Multi-Cloud

Terraform è particolarmente adatto all'orchestrazione di risorse fornite da diversi provider cloud noti. Utilizzando un'unica codebase scritta attraverso il linguaggio HCL, i team operativi possono gestire nativamente sistemi distribuiti in contemporanea su AWS, Microsoft Azure o Google Cloud Platform. Questo approccio cloud-agnostic snellisce i workflow aziendali riducendo l'esigenza di apprendere diversi framework di orchestrazione proprietari. Inoltre la flessibilità intrinseca di Terraform incoraggia la creazione di architetture cloud ibride, mitigando i vincoli architetturali legati a un unico fornitore.

7.3 Regole di Sicurezza Automatiche

Allo scalare dell'architettura è importante continuare a rispettare regole operative e requisiti aziendali. Le versioni più avanzate di Terraform permettono di definire controlli automatici tramite strumenti come HashiCorp Sentinel e Open Policy Agent. Questi strumenti analizzano in anticipo le modifiche previste da `terraform plan` e verificano che rispettino le regole definite ancora prima del rilascio. Ad esempio le policy possono impedire la creazione di server al di fuori delle aree consentite dal GDPR, oppure bloccare configurazioni troppo costose rispetto al budget del progetto.

In questo modo Terraform evita modifiche che potrebbero creare problemi di sicurezza o costi inutili.

7.4 Importazione di Infrastrutture Esistenti

Molte aziende che adottano l'IaC hanno già infrastrutture create in passato tramite modifiche manuali o configurazioni gestite direttamente dalle interfacce dei provider cloud. Questa transizione non costringe tuttavia a ritirare o smantellare intere architetture. Attraverso il comando `terraform import` [15], i server o componenti già attivi possono essere incorporati senza perdite all'interno dello stato centralizzato. In questo modo anche infrastrutture più vecchie possono essere integrate nei moderni processi di automazione, sicurezza e distribuzione continua.

8. Bibliografia

- [1] HashiCorp, *Terraform Documentation*, <https://developer.hashicorp.com/terraform/docs>
- [2] The Kubernetes Authors, *Kubernetes Official Documentation*, <https://kubernetes.io/docs/home/>
- [3] Kubernetes SIGs, *kind – Kubernetes IN Docker*, <https://kind.sigs.k8s.io/docs/>
- [4] HashiCorp, *Terraform Meta-Arguments: lifecycle*, <https://developer.hashicorp.com/terraform/language/meta-arguments/lifecycle>
- [5] HashiCorp, *Terraform Registry: Kubernetes Provider*, <https://registry.terraform.io/providers/hashicorp/kubernetes/latest/docs>
- [6] HashiCorp, *Terraform Modules*, <https://developer.hashicorp.com/terraform/language/modules>
- [7] HashiCorp, *Terraform Meta-Arguments: for_each*, https://developer.hashicorp.com/terraform/language/meta-arguments/for_each
- [8] HashiCorp, *Terraform Registry: Random Provider*, <https://registry.terraform.io/providers/hashicorp/random/latest/docs>
- [9] HashiCorp, *Terraform Registry: TLS Provider*, <https://registry.terraform.io/providers/hashicorp/tls/latest/docs>
- [10] HashiCorp, *Terraform Registry: Local Provider*, <https://registry.terraform.io/providers/hashicorp/local/latest/docs>
- [11] tehlers320, *Terraform Registry: Kind Provider*, <https://registry.terraform.io/providers/tehl3rs320/kind/latest/docs>
- [12] HashiCorp, *Terraform Data Sources*, <https://developer.hashicorp.com/terraform/language/data-sources>

- [13] HashiCorp, *Terraform Output Values*, <https://developer.hashicorp.com/terraform/language/values/outputs>
- [14] HashiCorp, *Terraform State Management e Remote State*, <https://developer.hashicorp.com/terraform/language/state>
- [15] HashiCorp, *Terraform Command: import*, <https://developer.hashicorp.com/terraform/cli/commands/import>